

DESIGN-QA GATE · SESSION REPORT

Day One. Real Bugs.

Installed a 53.8k-star deterministic detector, built the wrapper two skills already assumed existed, then found three real defects proving it — one that Impeccable itself missed.

CHAPTER 01 · THE PROBLEM

01

Every AI Coding Tool Ships The Same Slop

The Model Was Never The Problem

Every model trained on the same SaaS templates converges on the same tells, regardless of which agent generated them:

INTER / ROBOTO EVERYWHERE

PURPLE→BLUE GRADIENTS

CARDS NESTED IN CARDS

ICON TILES ABOVE HEADINGS

Impeccable ships 59 named rules for exactly this — deterministic pattern matches, not a model asked to have taste.

One Skill, Two Halves: Vocabulary Plus Detector

WHAT IT IS

A design vocabulary, not a prompt pack

- PRODUCT.md + DESIGN.md capture durable project context
- 23 commands: craft, critique, audit, polish, bolder, quieter...
- Every command reads project context before acting

STRUCTURED, NOT VIBES

HOW IT RUNS

Deterministic, not another model call

- 59 named rules, zero LLM, zero API key, zero token cost
- Regex, static-HTML, CSS-cascade, and visual-contrast engines
- Works with Claude Code, Codex, Cursor, Copilot, and more

INSTANT FEEDBACK

Two Tiers, So Editing Never Stalls

PER-EDIT · POSTTOOLUSE

Mechanical, unambiguous problems

- Broken images, clipped content, low contrast
- Gradient text, glow shadows, design-system drift

INTERRUPTS THE EDIT · 5S TIMEOUT

SESSION-END · STOP

Everything else, deduplicated once

- Copy cadence, palette and typography taste, layout rhythm
- Full rule set over every UI file touched this session

NEVER BLOCKS AN EDIT · 30S TIMEOUT

```
"PostToolUse": [{ "matcher": "Edit|Write|MultiEdit",  
  "hooks": [{ "command": "node .claude/skills/impeccable/scripts/hook.mjs" }] }]  
"Stop": [{ "hooks": [{ "command": "node .claude/skills/impeccable/scripts/hook.mjs" }] }]
```

CHAPTER 02 · WHAT WE BUILT

02

From Aspirational Docs To A Working Gate

Two Skills Already Assumed This Existed

`genspark-branded-deck` (line 212) and `marp` (line 283) both referenced `scripts/design-qa-detect.sh` and "the repo's pinned Impeccable wrapper" by exact path. Before this session, neither the skill nor the script existed on disk — same pattern as the `content-ideas-okf` cleanup earlier this session.

"The repo's pinned Impeccable wrapper" — true in intent, false in the filesystem, until today.

What Actually Landed This Session

COMPONENT	WHERE	STATUS
Impeccable skill	<code>.claude/</code> , <code>.agents/</code> , <code>.github/skills/</code>	Installed, one copy per harness
PostToolUse + Stop hooks	<code>.claude/settings.local.json</code> , <code>.codex/hooks.json</code> , <code>.github/hooks/</code>	Wired, verified
<code>design-qa-detect.sh</code>	<code>scripts/</code>	Built, Node $\geq$ 24 gate, pinned <code>3.5.0</code>
Version pins	<code>marp</code> , <code>genspark-branded-deck</code> SKILL.md	Fixed: stale <code>3.4.0</code> → verified <code>3.5.0</code>

REAL FINDINGS · FIRST RUN · ZERO CONFIGURATION

4

side-tab In **for-you-template.html**

A thick colored left-border stripe on outlier-accent cards — "the most recognizable tell of AI-generated UIs," per Impeccable's own rule text. Found in this repo's own daily-feed template, on the very first real run of the gate we'd just built.

CHAPTER 03 · THREE REAL BUGS

03

Every One Of These Was Found By Actually Running It

oversized-h1: A Sentence Set At Hero Size

Impeccable flagged this deck's own draft — a full-sentence headline at 112px dominates the slide with no room for anything else.

BEFORE — FLAGGED

"Impeccable Just Caught Its First Bug — In Our Own
Repo"
54 chars @ 112px

AFTER — CLEAN

"Day One. Real Bugs."
19 chars @ 112px

BUG 2 · IN LINT_PPTX.PY ITSELF, PRE-SESSION CODE

SLIDE_EMPTY Fired On Every Single Slide

Marp exports each slide's render as the slide's **background fill**, not a placed shape. `python-pptx`'s `slide.shapes` is genuinely `[]` — but `background.fill.type == PICTURE` proves real content exists. The pre-existing check only looked at `shapes`.

```
shapes = list(_iter_shapes(slide.shapes))
+ has_background_picture = slide.background.fill.type == MSO_FILL_TYPE.PICTURE
- if not shapes:
+ if not shapes and not has_background_picture:
    findings.append(Finding("SLIDE_EMPTY", "error", ...))
```

12 errors → 0. Verified a *genuinely* empty synthetic slide still correctly fires.

The Tool Missed White Text On White

Slide 8's status table rendered with body text at `#eef2f7` (near-white) against Marp's default light table background — nearly invisible. This deck's own `low-contrast` rule check **did not fire**. The detector's static-HTML contrast engine doesn't fully resolve Marpit's injected default table stylesheet against custom-themed `<td>` text color.

FOUND BY EYE, NOT BY THE GATE

FIXED WITH EXPLICIT TABLE/TH/TD RULES

The Honest Takeaway

WHAT THE GATE CAUGHT

2 of 3 bugs, automatically

- Our own oversized-h1, on the first export
- 4 side-tab findings in existing repo HTML

WORKING AS DESIGNED

WHAT IT DIDN'T

Table contrast, and a PPTX-side gap

- White-on-white table text — Impeccable's own miss
- SLIDE_EMPTY blind spot — pre-existing, unrelated code

A GATE, NOT A GUARANTEE

CHAPTER 04 · THE BOUNDARY

04

Two Detectors, One Line You Don't Cross

Never Point Impeccable At A .pptx

IMPECCABLE

HTML surfaces only

- .html, .css, .tsx, .jsx, .vue, .svelte...
- presentation, for-you-template.html, Genspark HTML capture
- Does not understand native .pptx at all

NEW THIS SESSION

LINT_PPTX.PY

Native PowerPoint, already mature

- 15 rules: overflow, overlap, contrast, DPI, layout repetition
- Its own ignore_rules / waivers mechanism, already in use
- Not replaced, not duplicated — just documented as the boundary

ALREADY BUILT, PRE-SESSION

Run It Every Time

design-qa-detect.sh is now live in marp and genspark-branded-deck.
This deck is the proof: built, broken, caught, fixed, and re-verified —
three times — before delivery.